C Security

Martin Johns

The C programming language

Developed 1972 by Dennis Ritchie

All major operating systems are written in C or C variants

- Windows
- Linux
- Mac Os X, Solaris, *BSD, ...

Most popular applications

- Microsoft Office
- Internet Explorer, Firefox
- Photoshop, iTunes, ...

Important Servers

- Web Server (Apache, IIS)
- Mail Server, DNS Server, SSH, Telnet, ...

The C programming language

C is a “low level” programming language

- Procedural
- No object-oriented features
- Pointers

C is compiled into machine/binary code

No memory abstraction

- All variables in C programs correspond directly to specific location in the RAM

C successors

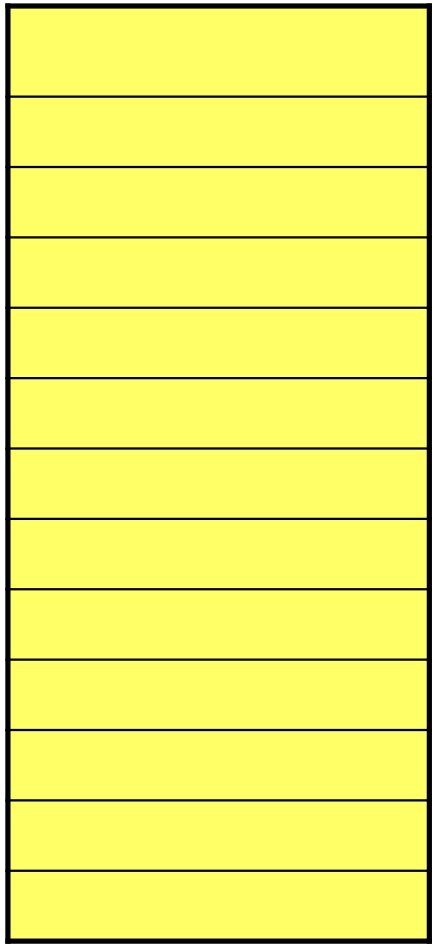
C++

- Developed 1979 von Bjarne Stroustrup
- Superset of C
- Object-oriented
- Tons of advanced/complex features
  - Templates, operator overloading, polymorphism, multiple inheritance
  - But no memory safety
- In heavy use at Microsoft

Objective C

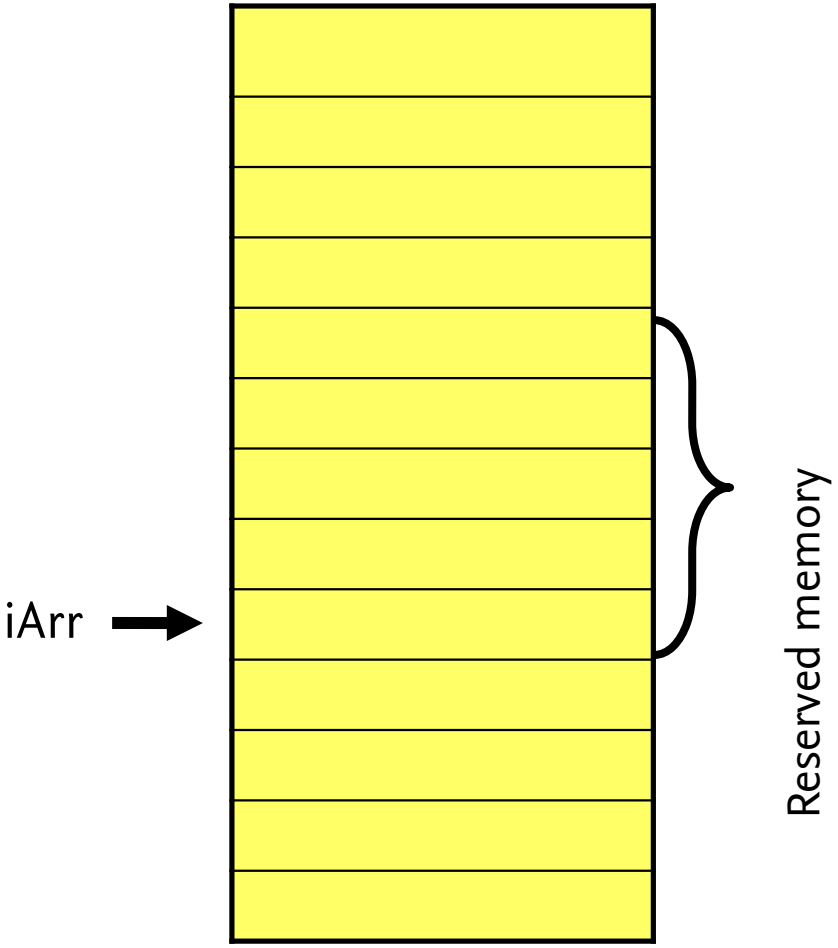
- Also an object-oriented superset of C
- Small talk like message passing
- Dynamic typing
- But no memory safety
- In heavy use at Apple / iOS

Memory management



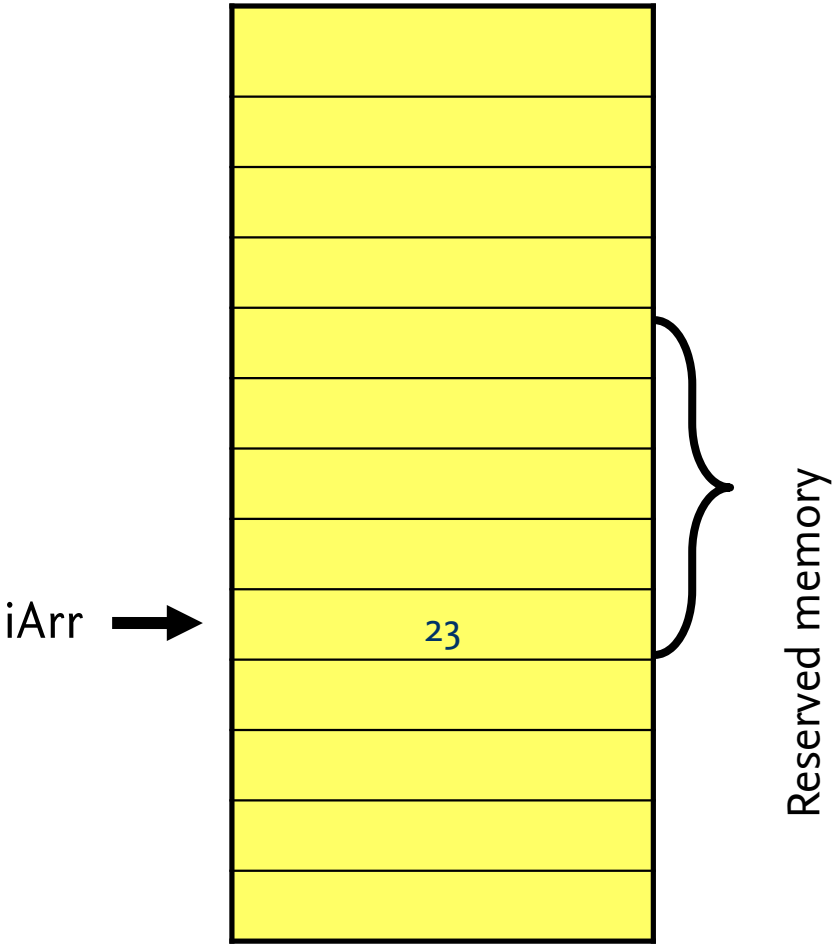
```
int iArr [5];
```

Memory management



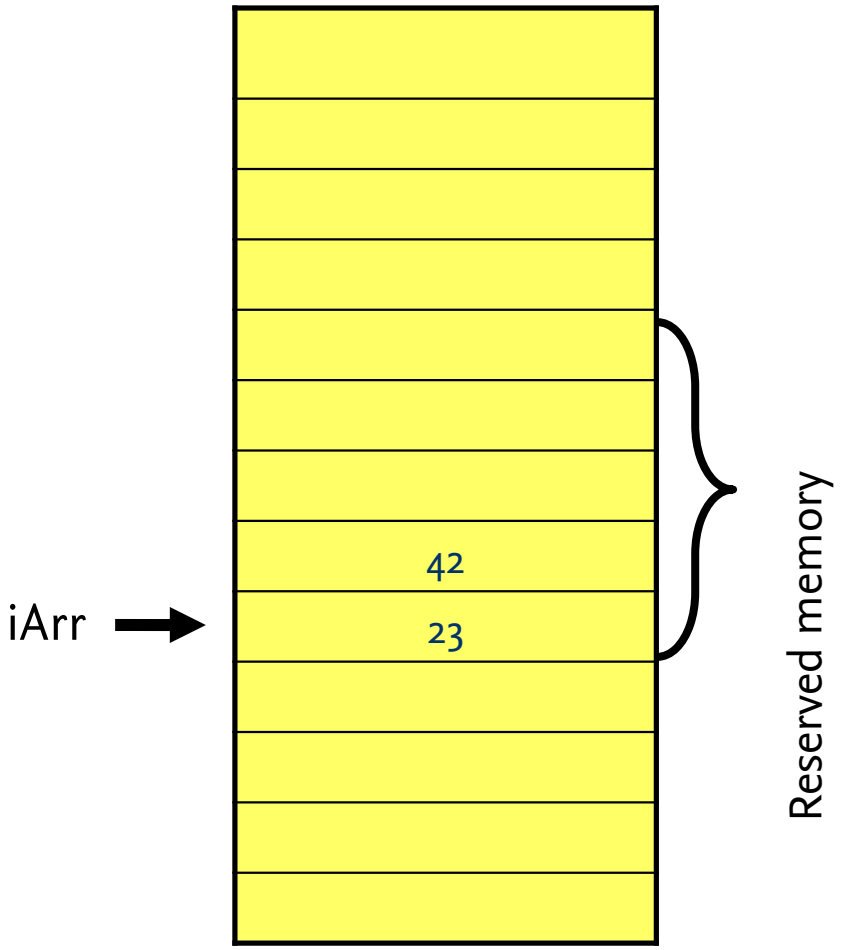

```
int iArr [5];
iArr[0] = 23;
```

Memory management



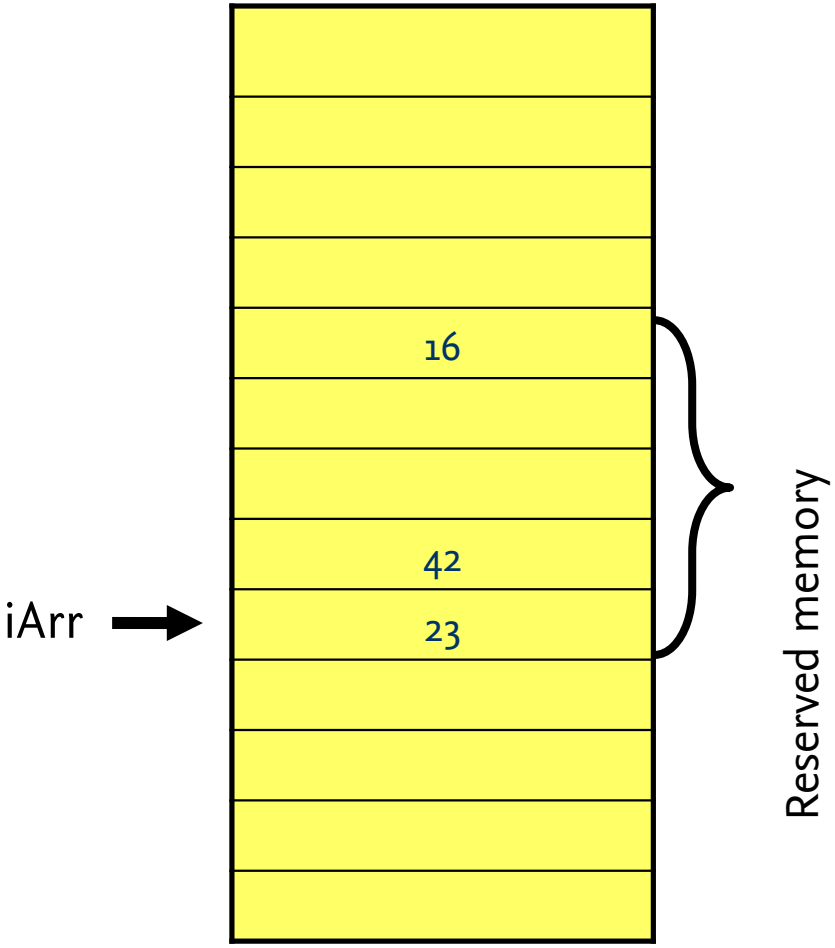

```
int iArr [5];
iArr[0] = 23;
iArr[1] = 42;
```

Memory management




```
int iArr [5];
iArr[0] = 23;
iArr[1] = 42;
iArr[4] = 16;
```

Memory management



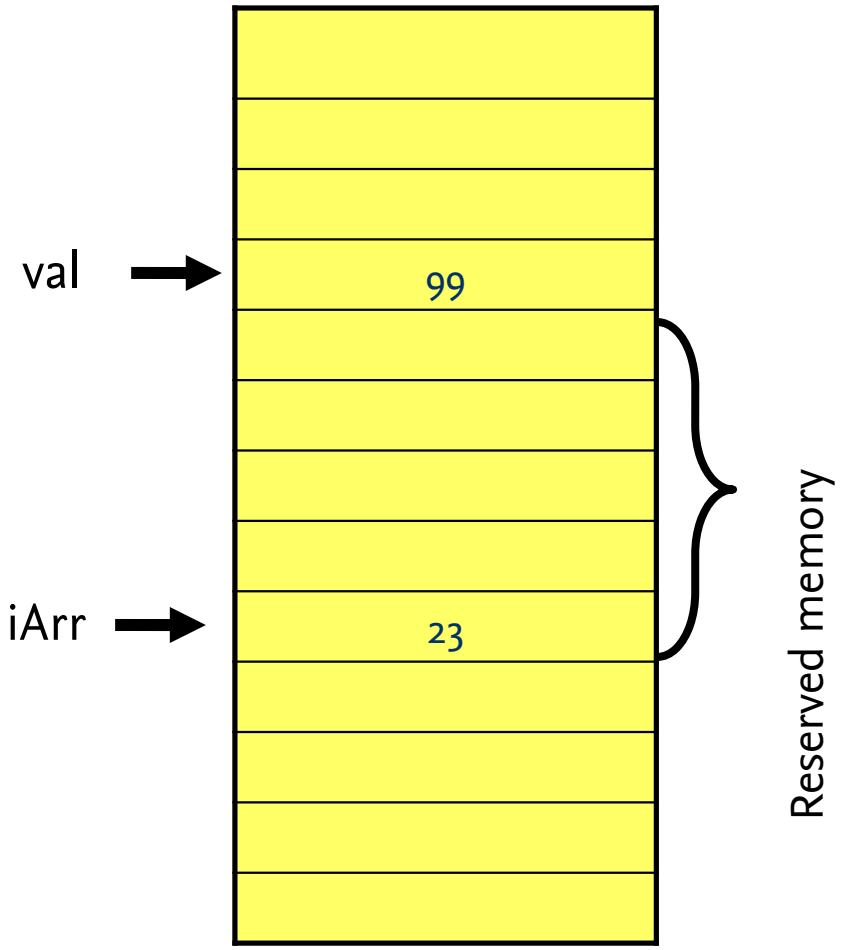

```
int iArr [5];
iArr[0] = 23;
iArr[1] = 42;
iArr[4] = 16;
```

Buffer Overflow

The buffer boundaries are not enforced!

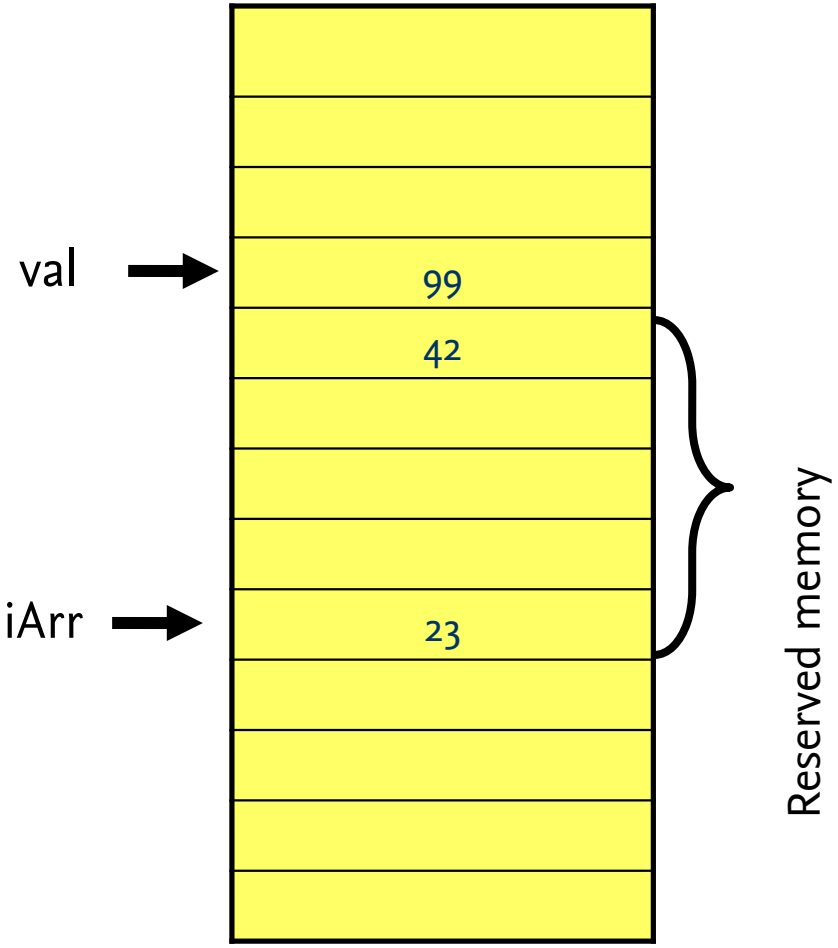
what happens if a misbehaving program violates them?

Memory management



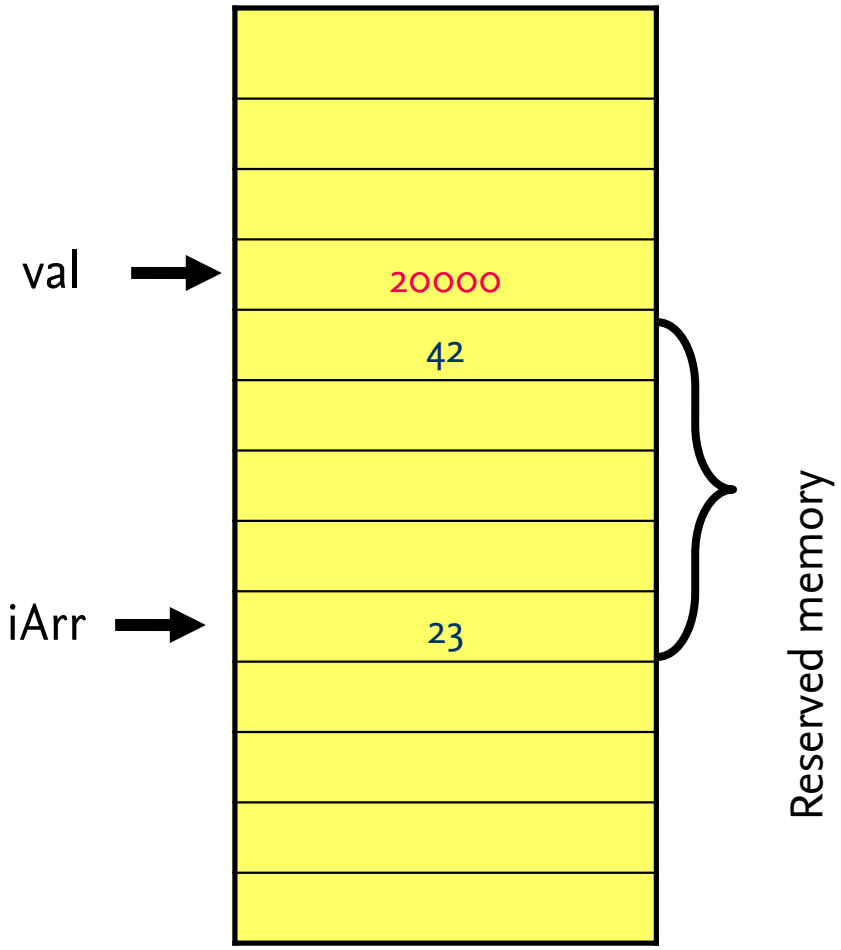

```
int val = 99;
int iArr [5];
iArr[0] = 23;
iArr[4] = 42;
```

Memory management




```
int val = 99;
int iArr [5];
iArr[0] = 23;
iArr[4] = 42;
iArr[5] = 20000;
```

Memory management




```
int val = 99;
int iArr [5];
iArr[0] = 23;
iArr[4] = 42;
iArr[5] = 20000;
```

Machine Code

Executed directly by the processor

Simple instruction format:

Do *A* with *B* and store the result in *C*

- Assignments
- Jumps
- Conditionals

No variables / types

- Instead: Memory locations
  - Registers
  - RAM
- Memory locations are addressed
  - Absolute
  - Relative
  - In both cases some absolute values are needed

Subroutines

A function in the C code is translated to a subroutine in machine code

Instructions

- Jump to subroutine at address *X*
- return

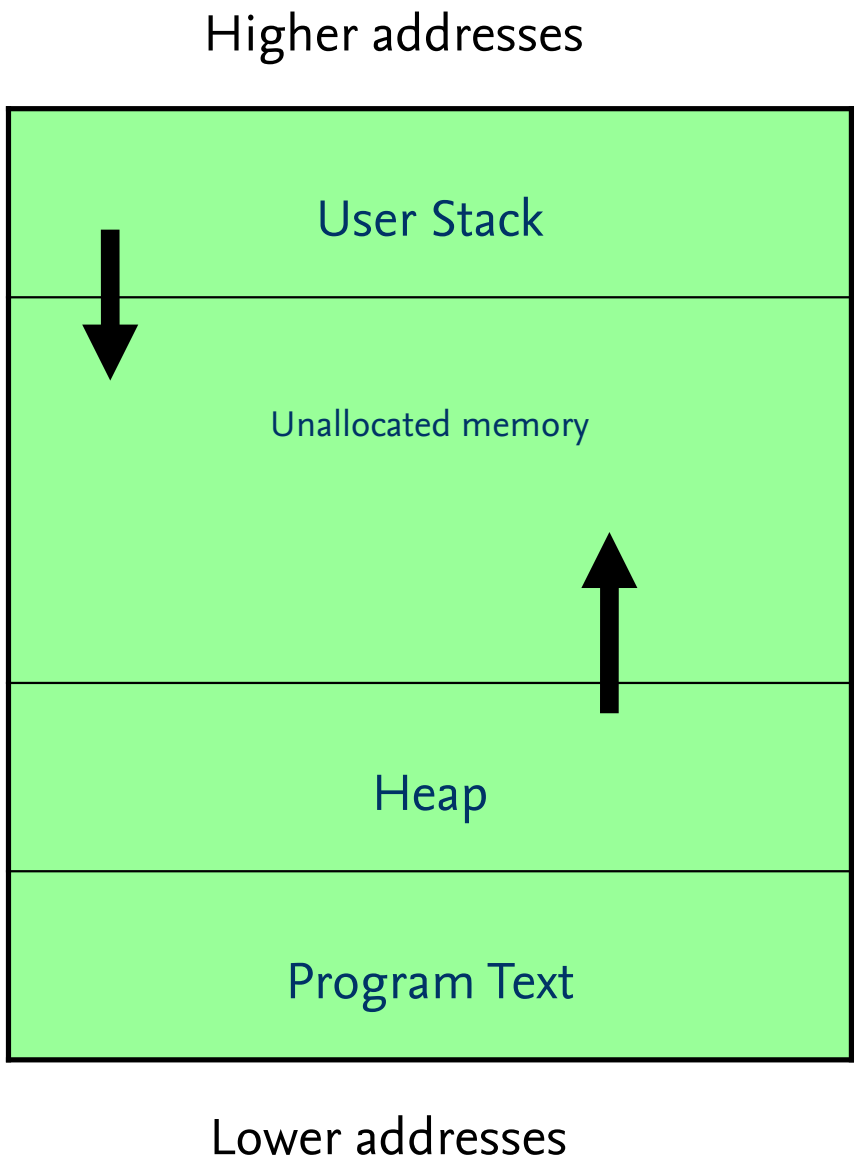
At compilation-time it is unknown how often a given subroutine is called

- How much memory should be reserved?
- How are the local values addressed?
- Where should the execution return after the subroutine has finished?

The Stack

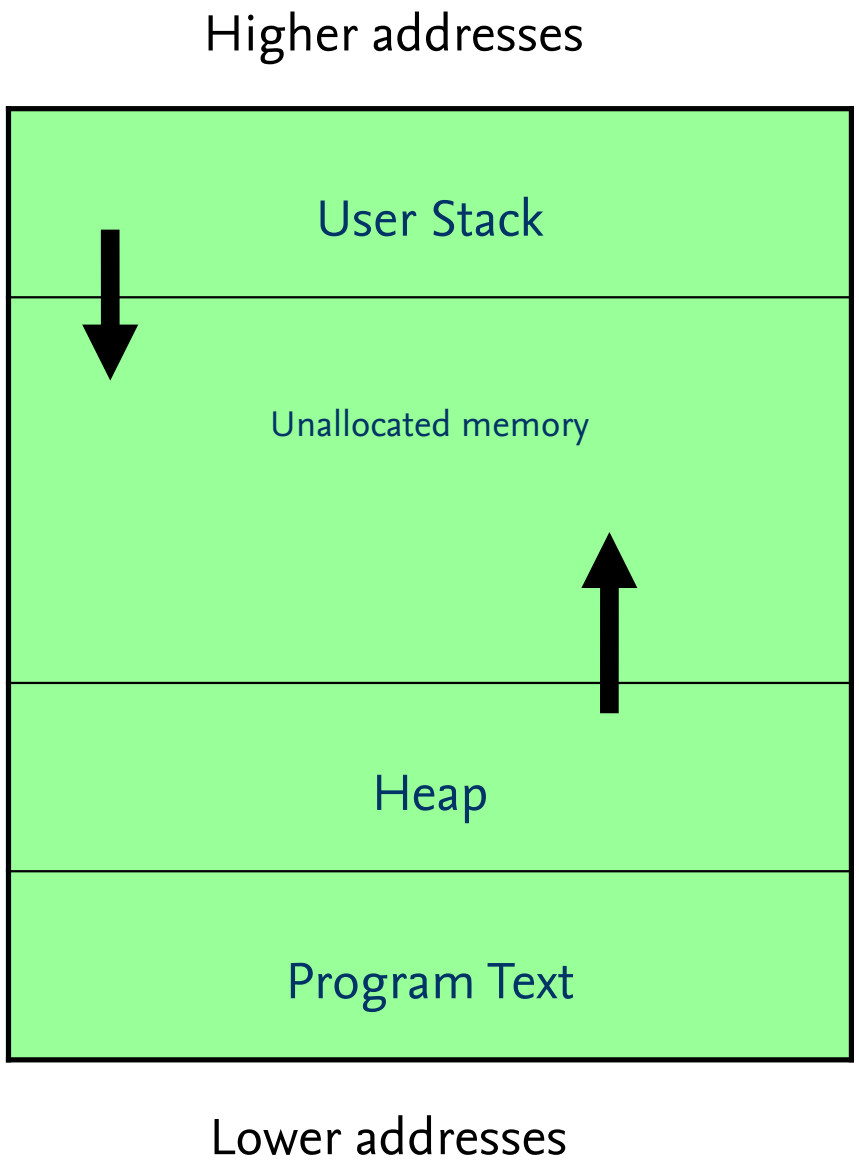
- A dedicated memory area
- Holds dynamically data which is created
  - Local variables
  - Execution-context information

Basics: Process Memory Layout



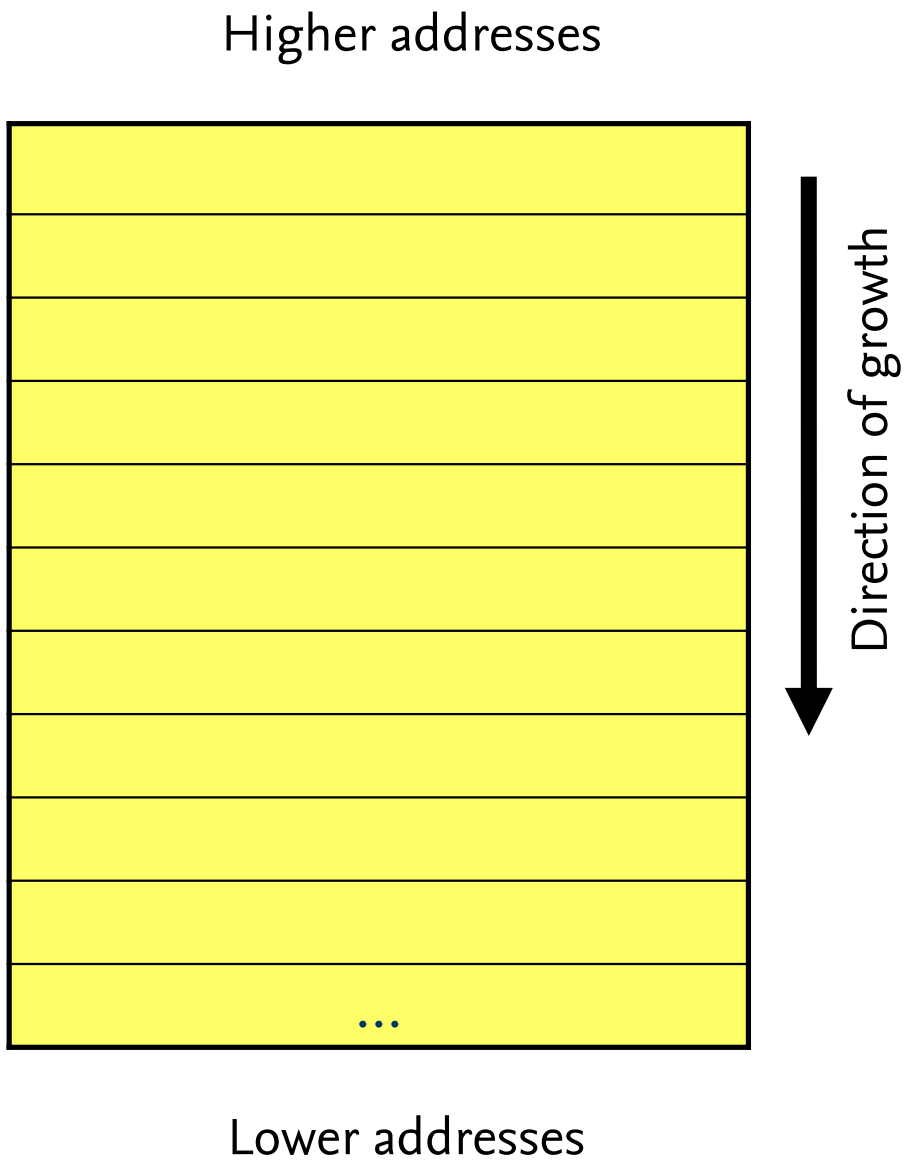
- Depending on the operating system the actual memory layout may somewhat differ
  - Usage of memory below the heap
  - Usage of memory above the stack
- Coherent memory is always read and written from lower addresses to higher addresses (not considering little/big endian)

Basics: The Stack vs. The Heap

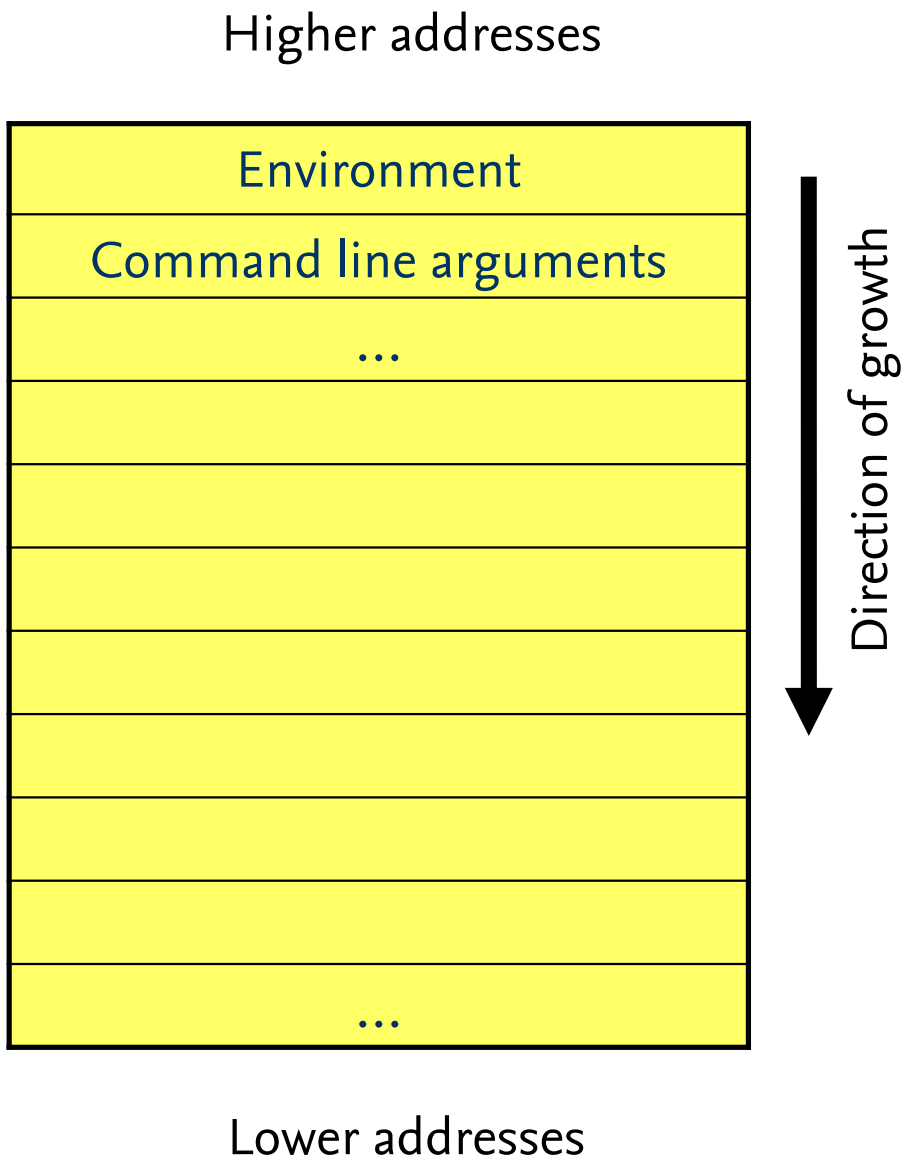


- The Stack grows from the higher to the lower addresses
- The Heap (initially) grows from the lower to the higher addresses
- The Stack is used to store data that is local to functions
- The Heap is used to store data that is persistent between function calls
- Stack allocation is implicit
  `char test[42];`
- Heap allocation is (usually) explicit
  `char *test = malloc(42*sizeof(char));`

Basics: The Stack



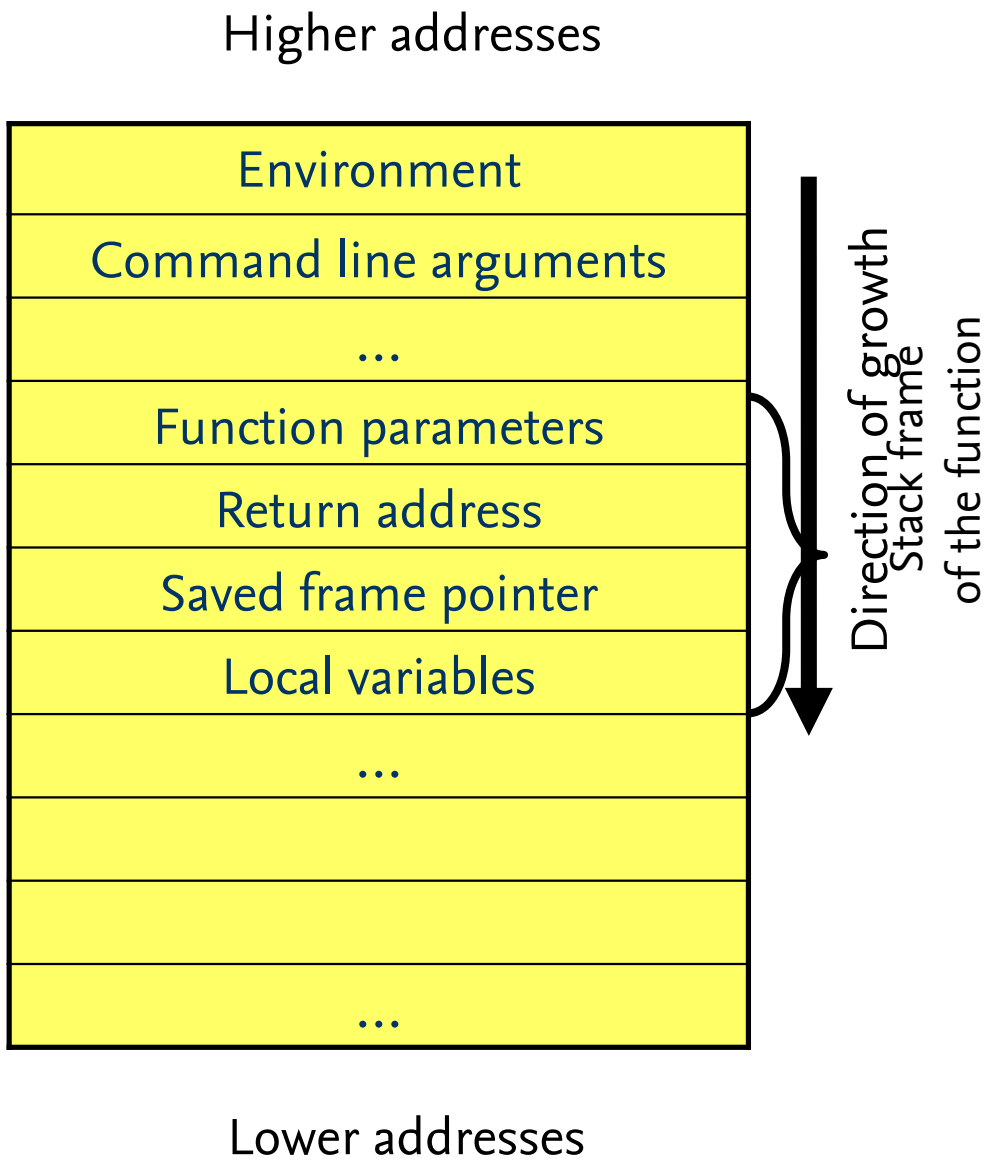
Basics: The Stack



The Stack is used for:

- The Environment
- Command line arguments

Basics: The Stack



The Stack is used for:

- The Environment
- Command line arguments
- Function parameters
- Return addresses
- Saved frame pointers
- Local variables

Basics: The Stack – Processor view (x86)

Several processor inherent constructs define the Stack

- The register SP points to the top of the stack
  - Instructions that are stack-aware modify the value of SP automatically
    - push, pop
    - call, return
    - (enter, leave)

The Frame Pointer

- The Frame Pointer is utilized by a function to access the local function context on the stack (function parameters, local variables)
- The register BP is used as Frame Pointer (assigned by the compiler)
  - Instructions to modify this register are (originally) generated by the compiler
  - Local variables and function parameters are referenced relatively to the Frame Pointer
    - Function parameters have positive offsets
    - Local variable have negative offsets

The stack

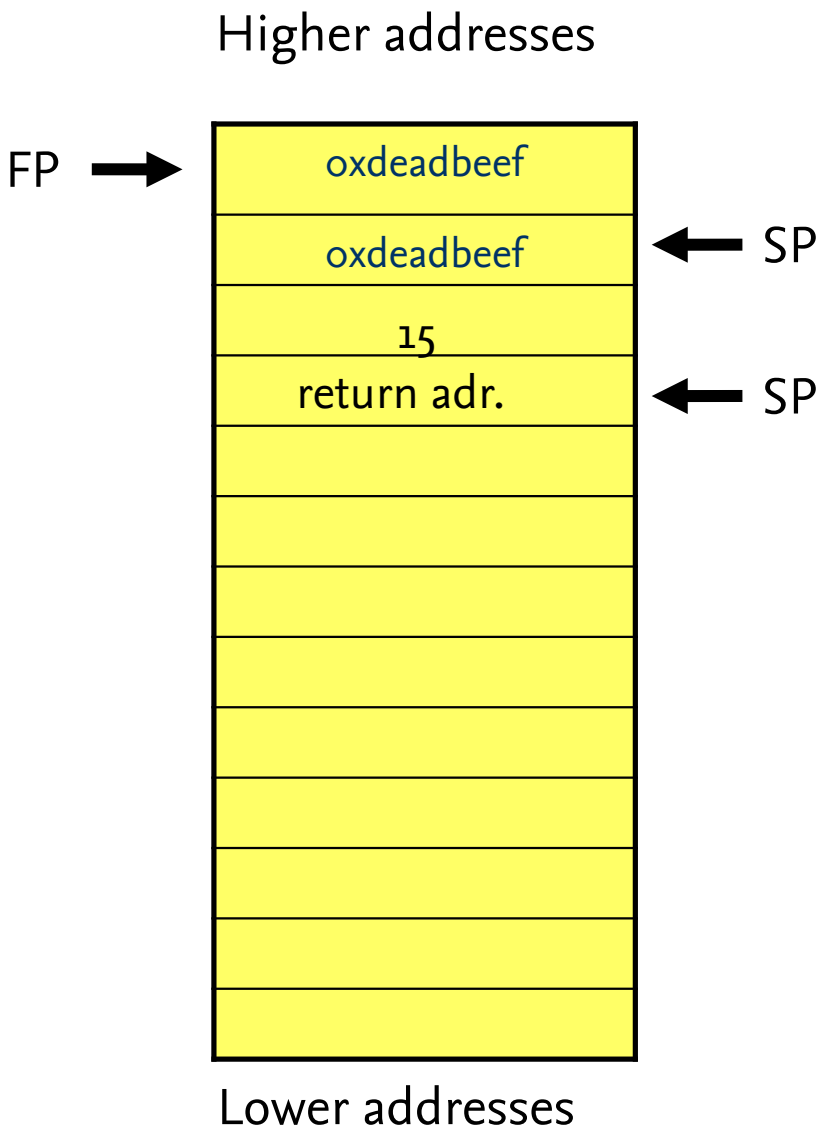
Instructions:

- push(val): Put val on top of the stack
- pop: Get the value that is on top of ths stack

Important registers:

- IP: Points to the next instruction
- SP: Points to the top if the stack
- FP: Defines the local data-context of a function on the stack
  - All local memory-addresses of the function are relative to FP

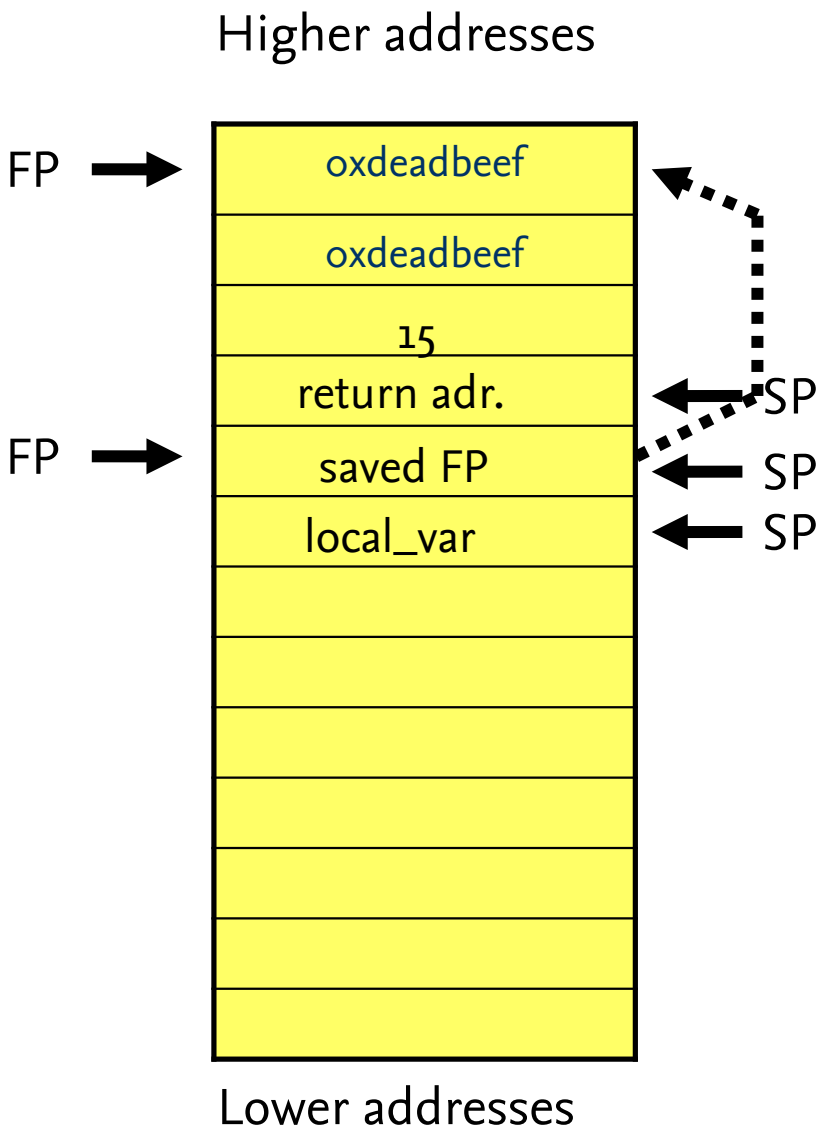
Basics: The function call




```
b = something(15);
```

- Move SP to reserve space for function parameter
- Write value of function parameter on the stack
- Call function (implicitly moves return address on the stack)

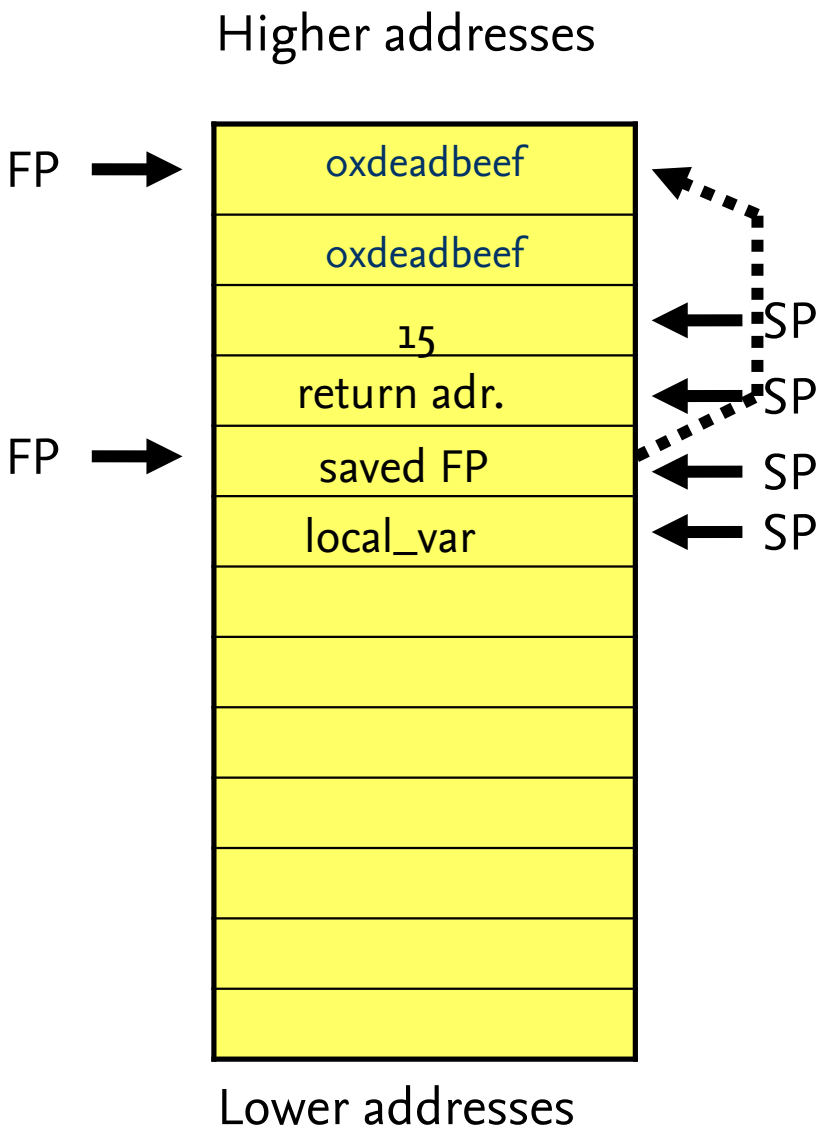
Basics: The function prolog




```
int something(int para){
   int local_var;
   ...
}
```

- Save the old FP on the stack
- Move the FP to the new frame
- Move the SP to reserve space for the local variables

Basics: The function epilog

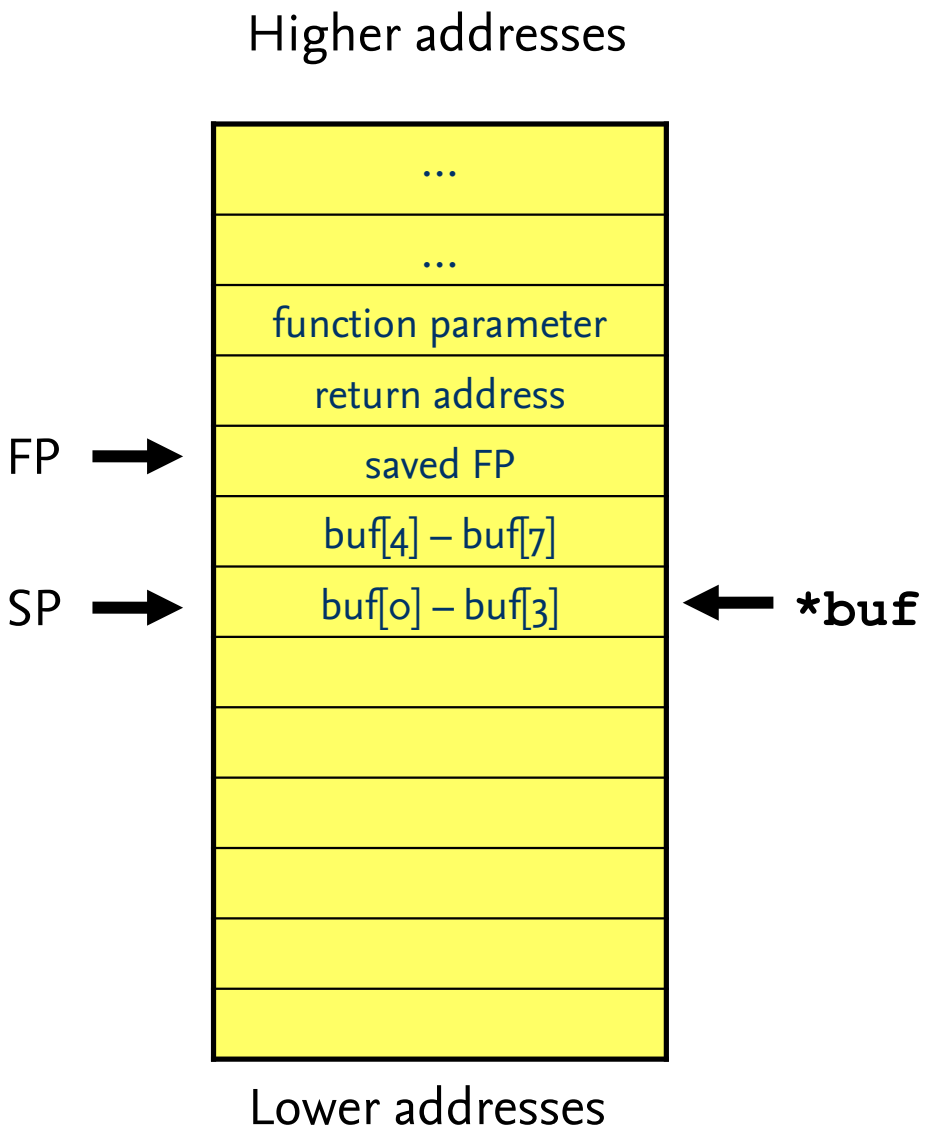



```
int something(int para){
   ...
   return (0);
}
```

- Write the return value to the register AX
- Move the SP back to the FP
- Restore the old FP
- Pop the stored old FP
- Pop the return address and leave the function

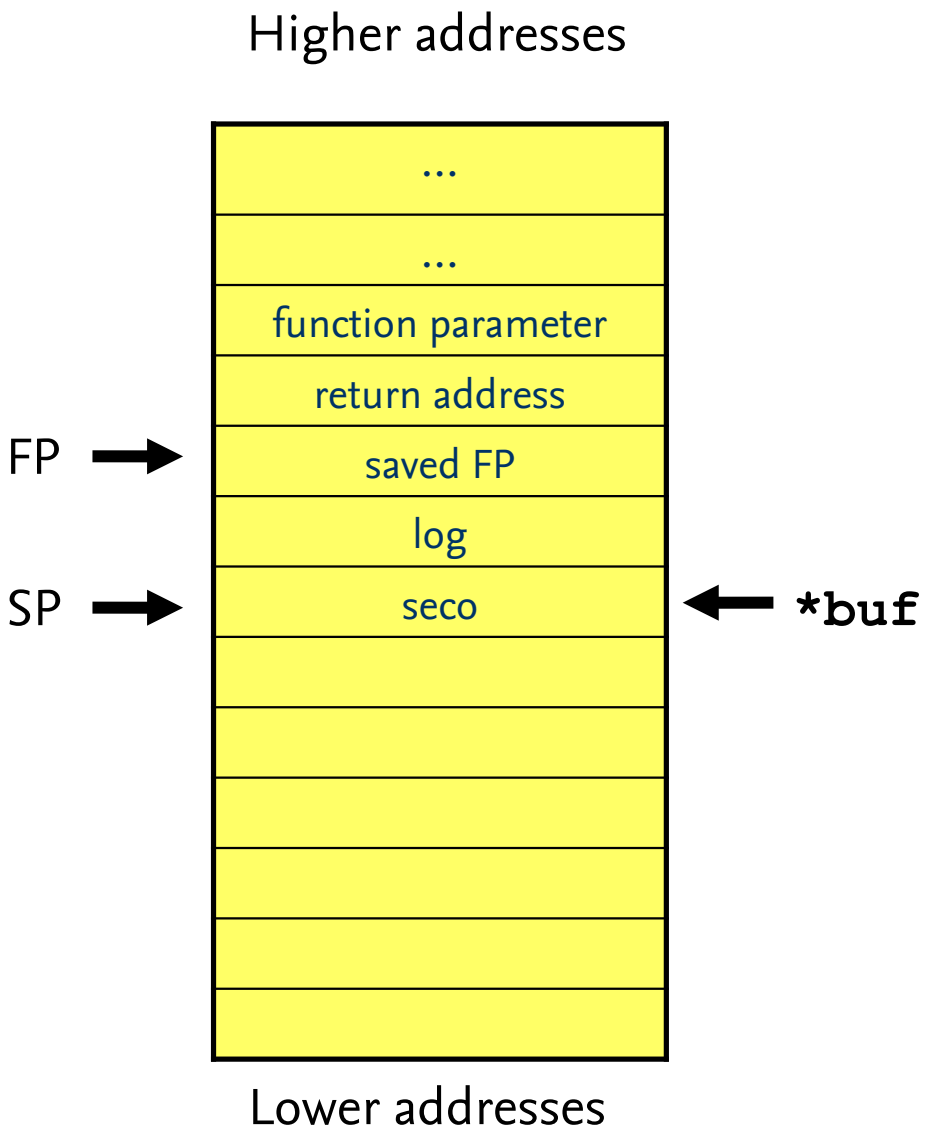
Buffer Overflows on the stack

Buffer Overflow on the stack



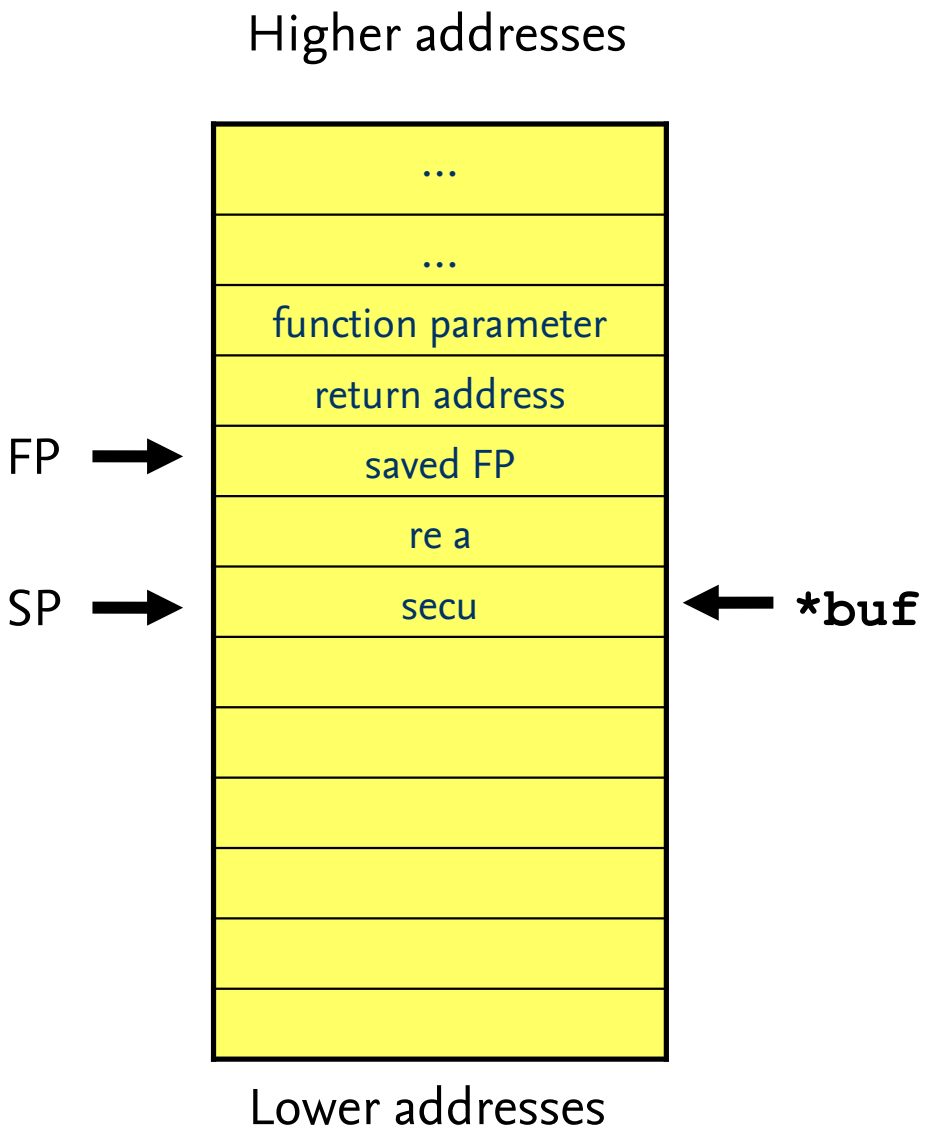

```
int something(int para){
  char buf[8];
  ...
  strcpy(buf, "secolog");



}
```

Buffer Overflow on the stack




```
int something(int para){
  char buf[8];
  ...
  strcpy(buf,
  "secolog"); ...
  strcpy(buf,
         "secure
  analysis");
}
```

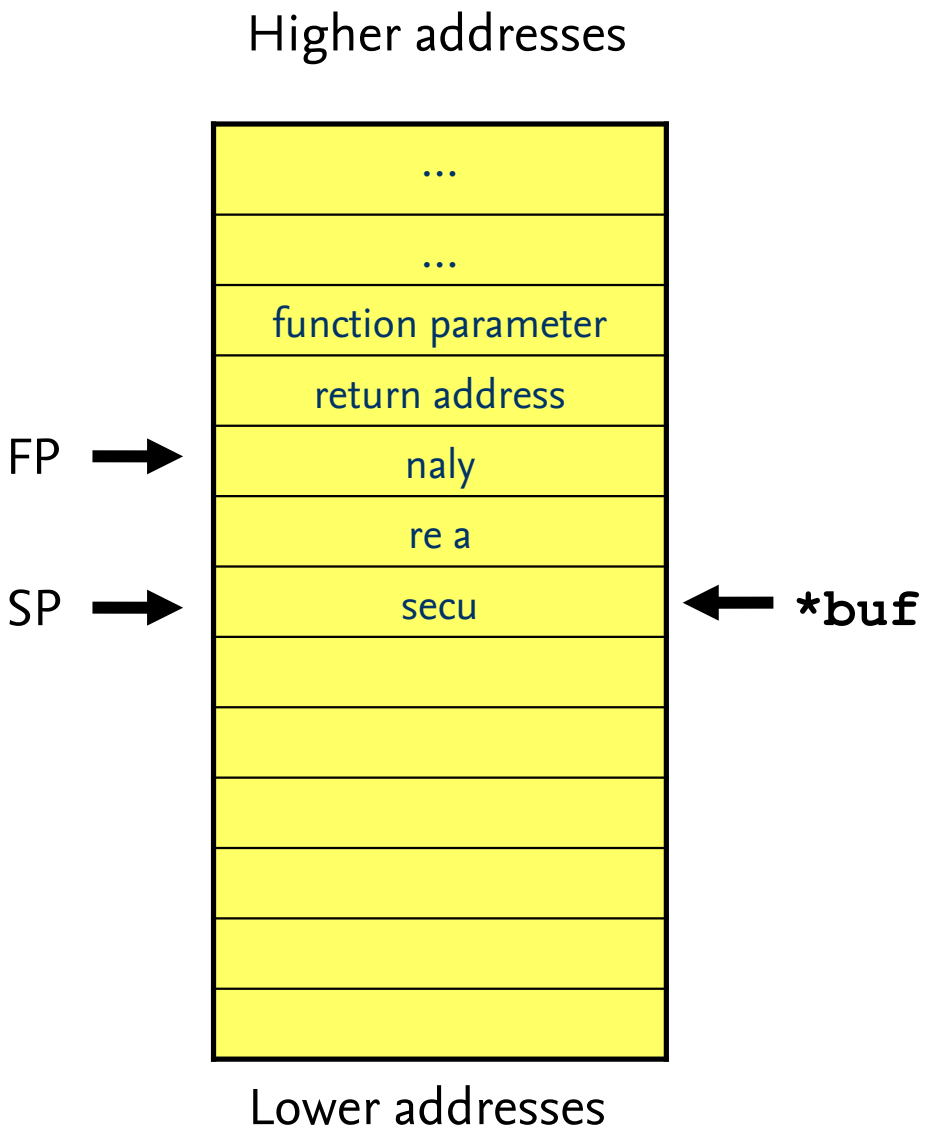
Buffer Overflow on the stack




```
int something(int para){
  char buf[8];
  ...
  strcpy(buf,
  "secolog"); ...
  strcpy(buf,
         "secure
  analysis");
}
```

- The saved Frame Pointer gets overwritten

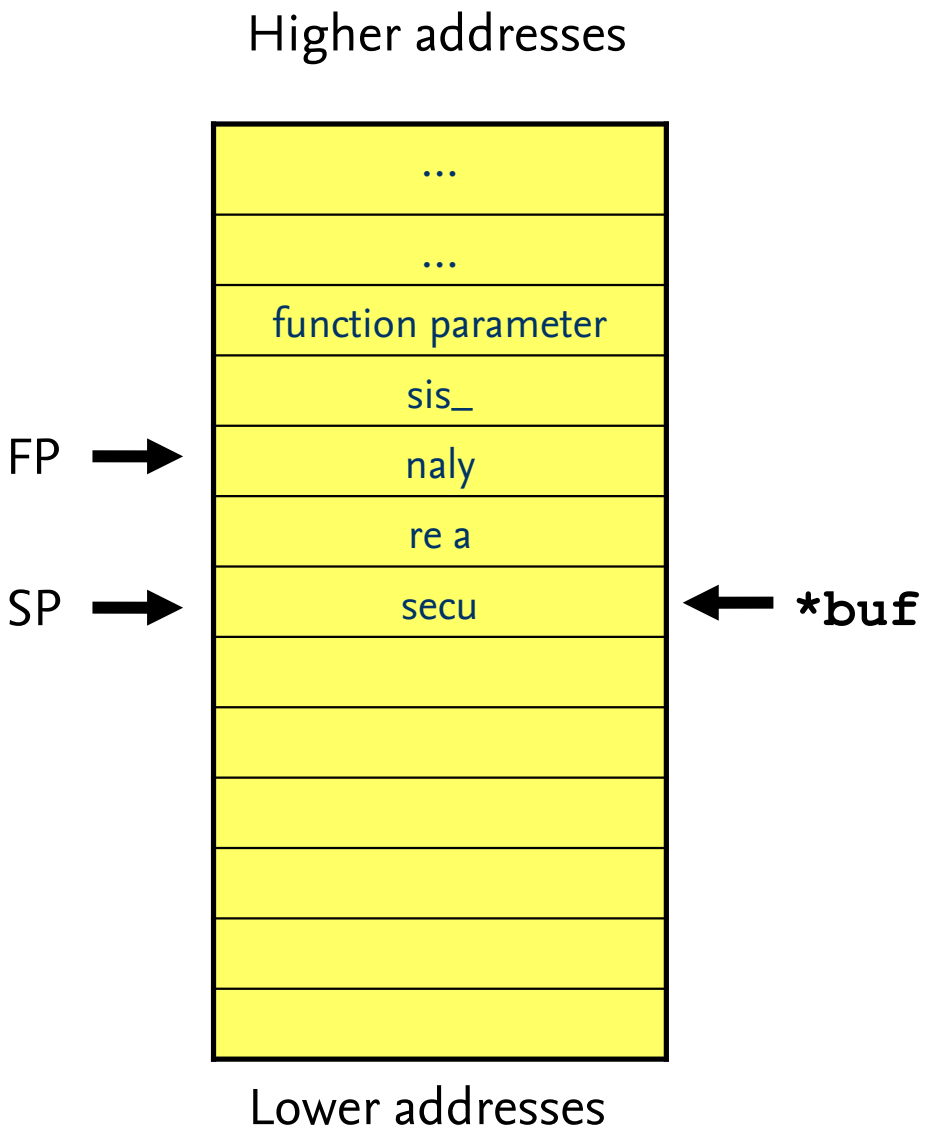
Buffer Overflow on the stack




```
int something(int para){
  char buf[8];
  ...
  strcpy(buf,
  "secolog"); ...
  strcpy(buf,
         "secure
  analysis");
}
```

- The saved Frame Pointer gets overwritten
- The return address gets overwritten

Buffer Overflow on the stack



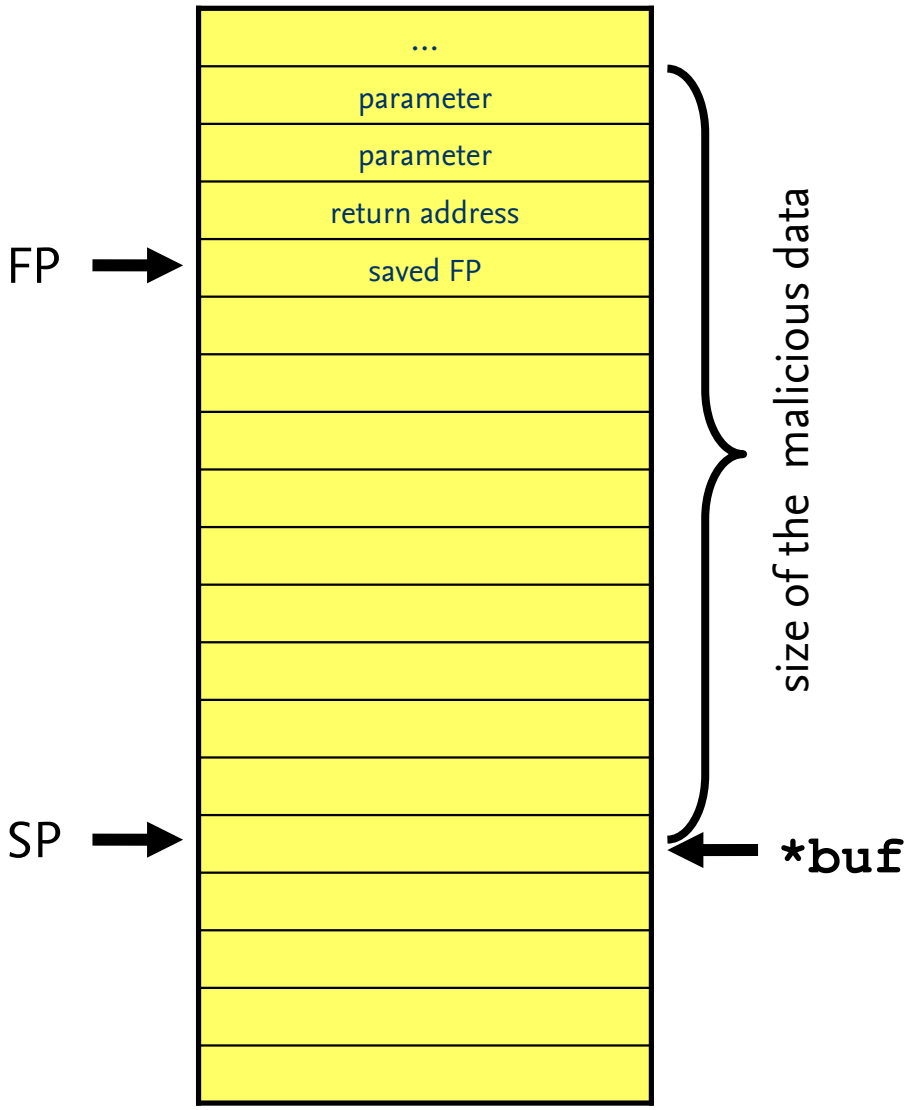

```
int something(int para){
  char buf[8];
  ...
  strcpy(buf,
  "secolog"); ...
  strcpy(buf,
         "secure
  analysis");
}
```

- The saved Frame Pointer gets overwritten
- The return address gets overwritten

Using a stack based Buffer Overflow to hijack execution

- Construct a buffer that contains executable code (the shellcode)
- Guess the memory address of the vulnerable buffer on the stack
- Pad the constructed shellcode
  - Before the shellcode: As many NOP instructions as possible
  - Behind the shellcode: The guessed address of the beginning of the shellcode (or the nops)
  - The constructed buffer has to be big enough to overwrite the return address on the stack

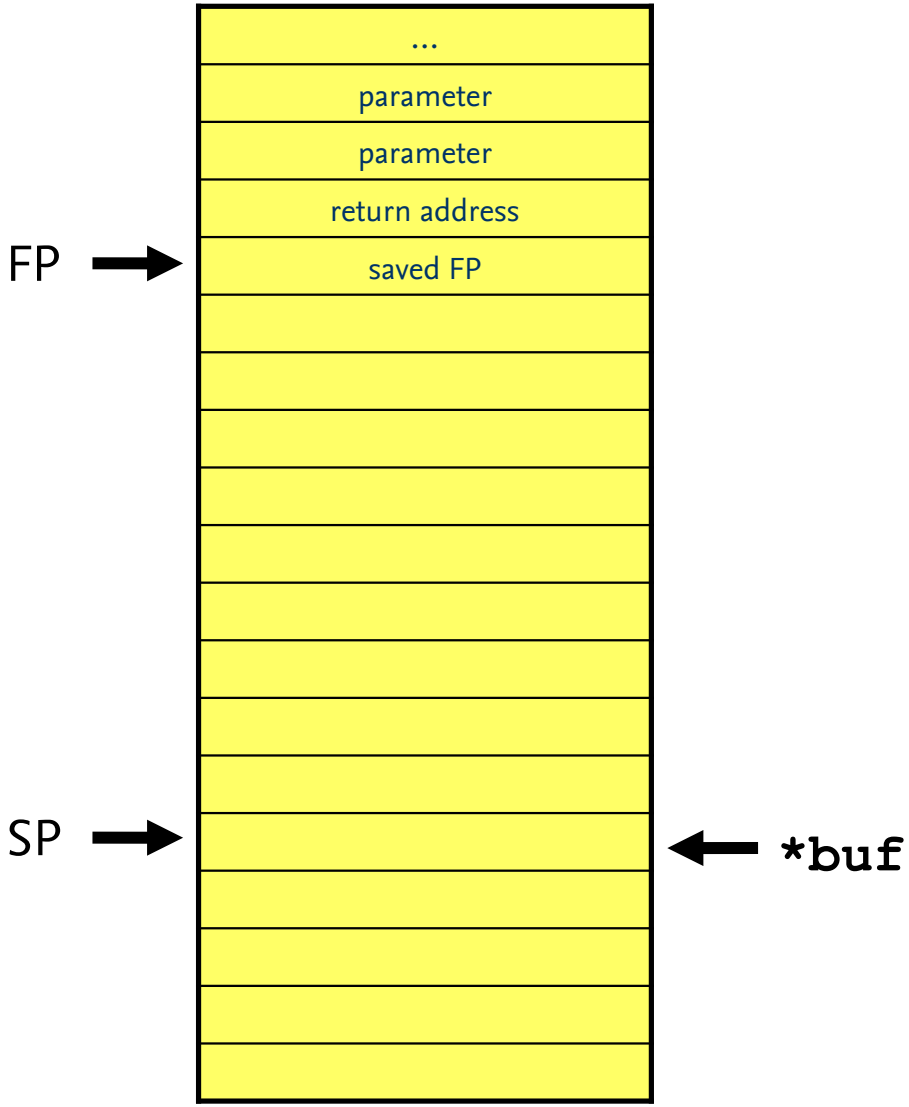
Buffer Overflow on the stack




```
int something(int,int){
   char buf[512];
   ...
   strcpy(buf,argv[1]);
}
```

- argv[1] contains the constructed malicious data
- The malicious data is big enough to overflow the buffer and overwrite the rerturn address

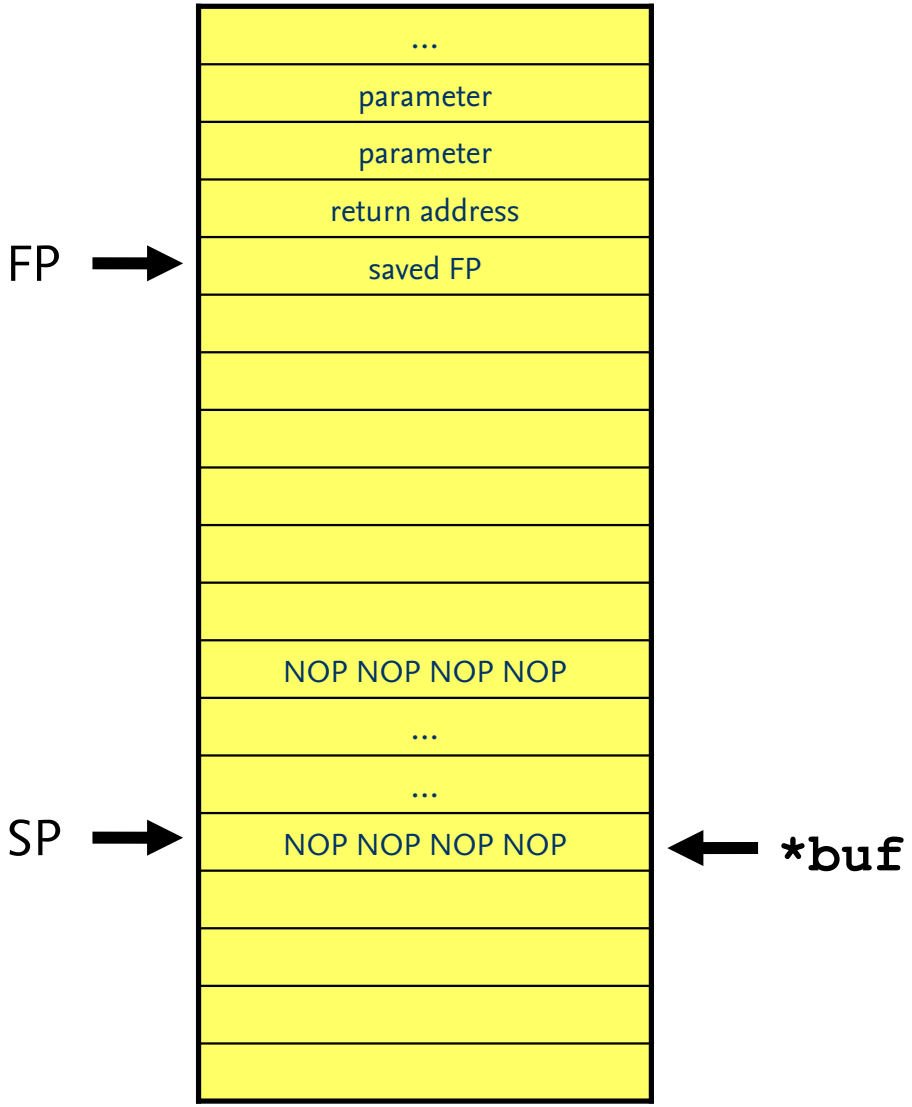
Buffer Overflow on the stack




```
int something(int,int){
   char buf[512];
   ...
   strcpy(buf,argv[1]);
}
```

- argv[1] contains the constructed malicious data
- The data starts with the NOPs

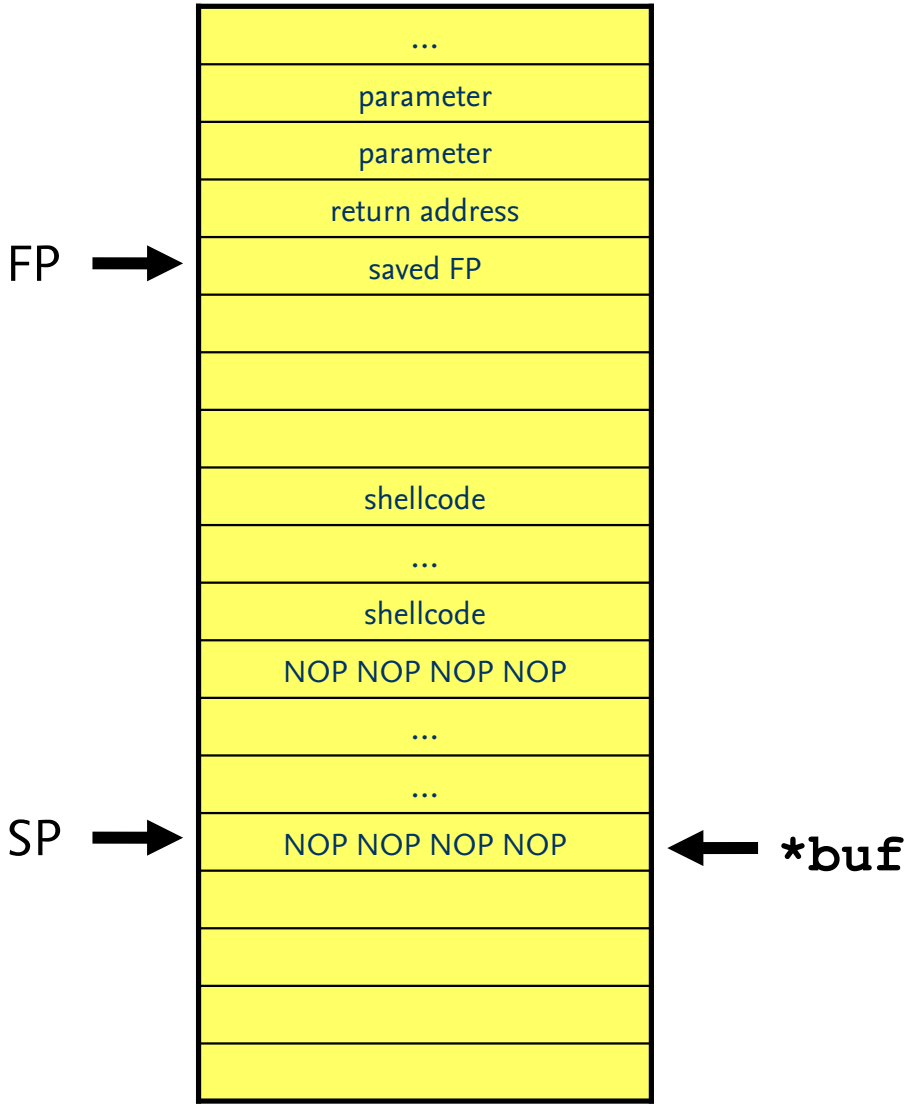
Buffer Overflow on the stack




```
int something(int,int){
  char buf[512];
  ...
  strcpy(buf,argc[1]);
}
```

- argc[1] contains the constructed malicious data
- The data starts with the NOPs
- In the middle follows the Shellcode

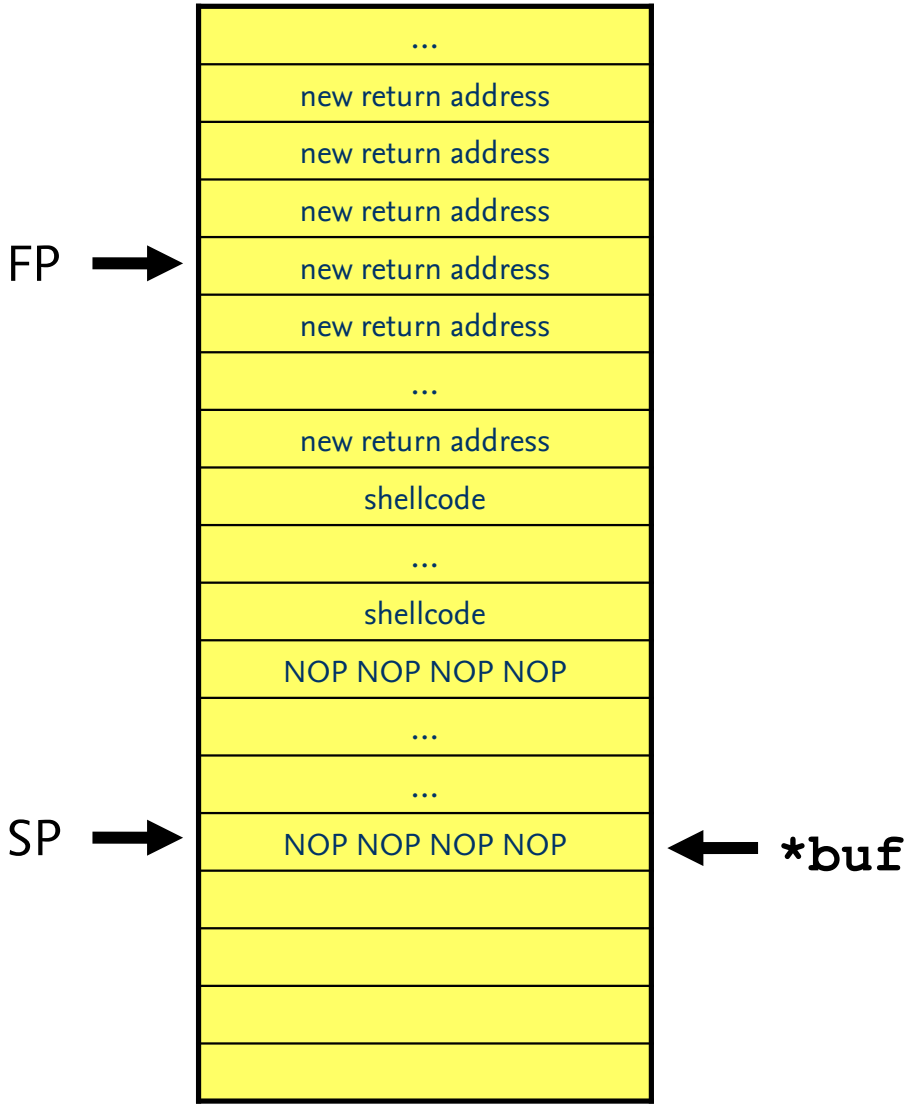
Buffer Overflow on the stack




```
int something(int,int){
   char buf[512];
   ...
   strcpy(buf,argc[1]);
}
```

- argc[1] contains the constructed malicious data
- The data starts with the NOPs
- In the middle follows the Shellcode
- The remaining buffer is filled with the (guessed) malicious return address

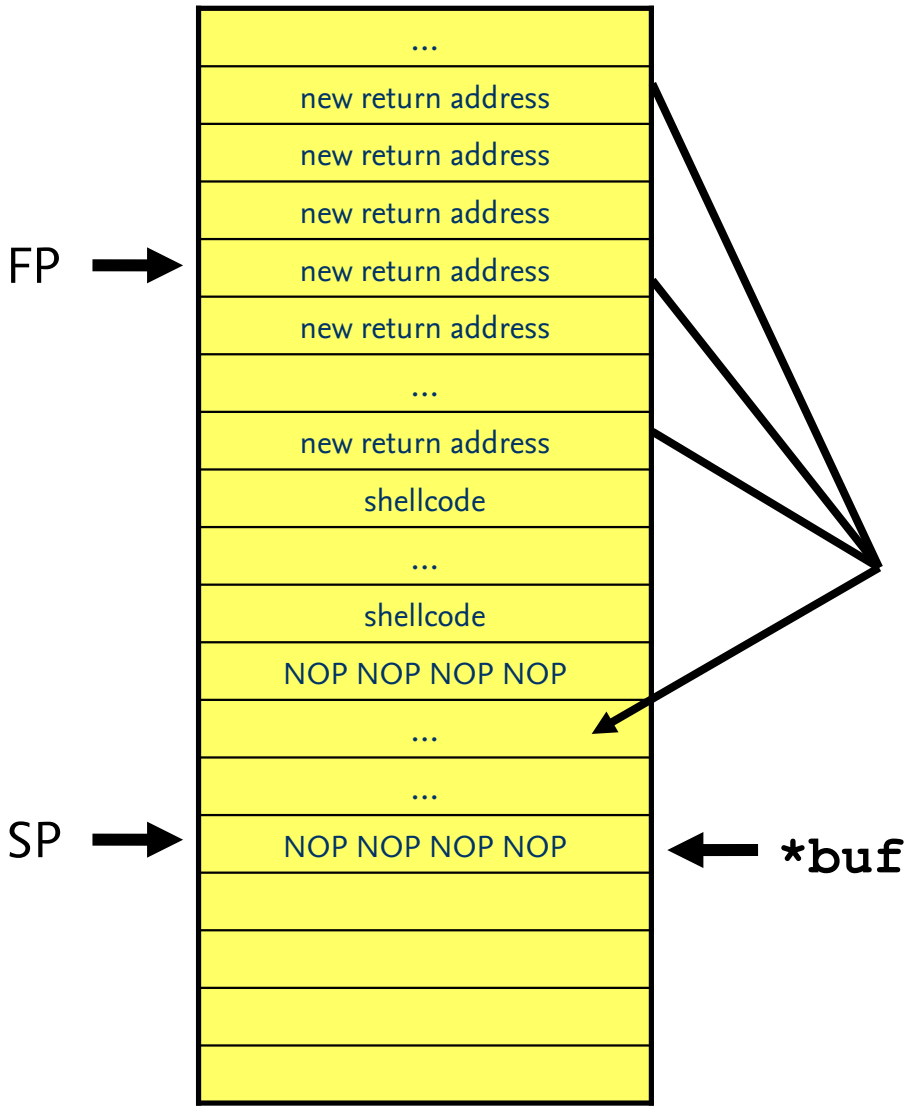
Buffer Overflow on the stack




```
int something(int,int){
   char buf[512];
   ...
   strcpy(buf,argc[1]);
}
```

- argc[1] contains the constructed malicious data
- The data starts with the NOPs
- In the middle follows the Shellcode
- The remaining buffer is filled with the (guessed) malicious return address

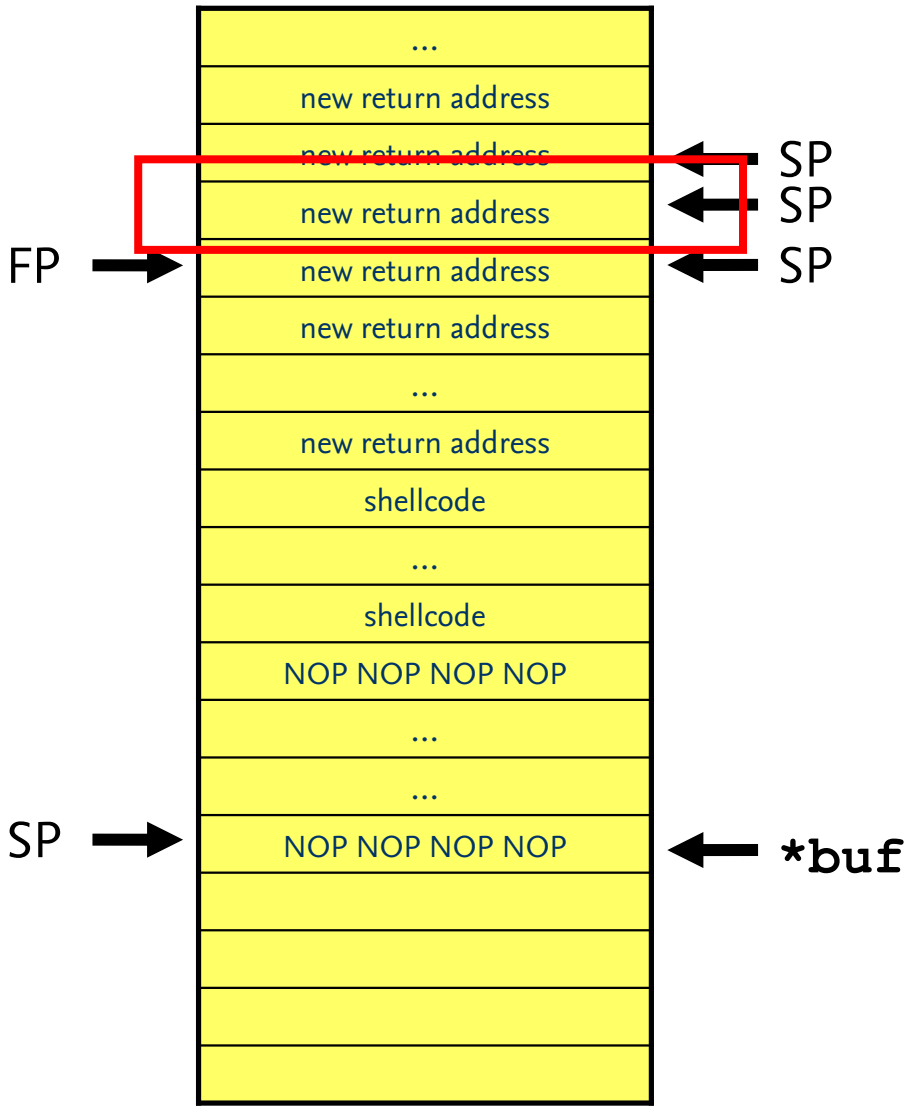
Buffer Overflow on the stack




```
int something(int,int){
   char buf[512];
   ...
   strcpy(buf,argc[1]);
}
```

- The new return address points to a location in the buffer
- At least if the guessed location of the stack was correct

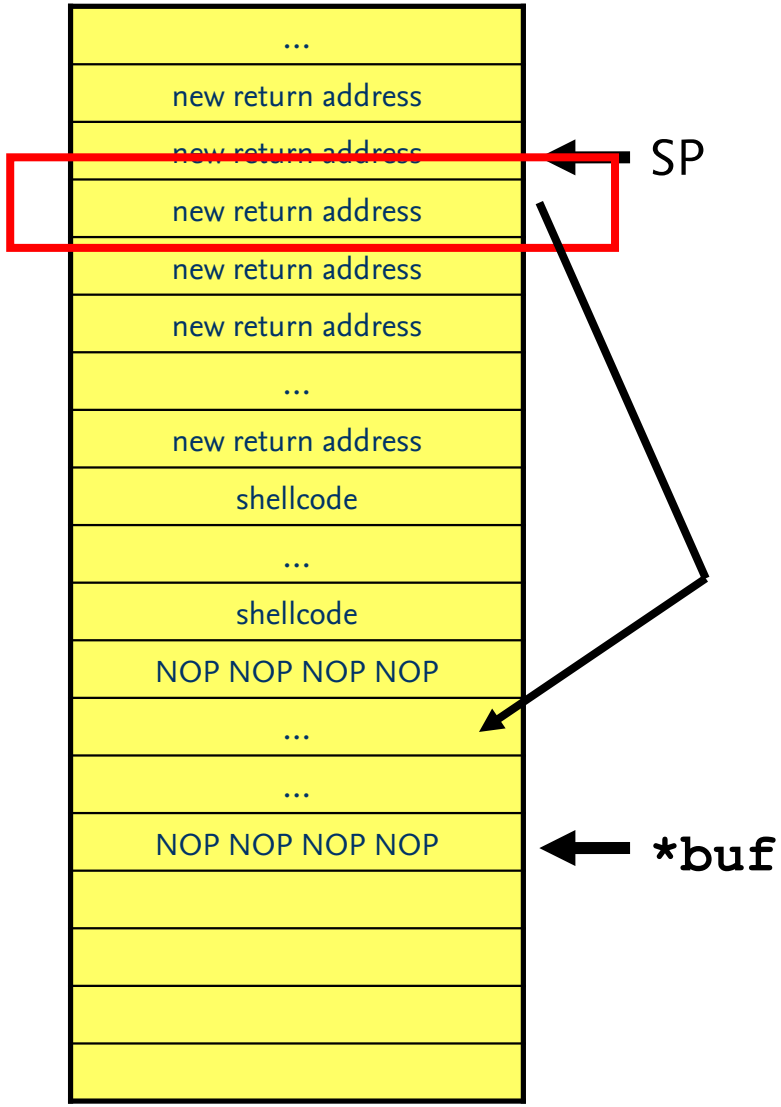
Buffer Overflow on the stack




```
int something(int,int){
  char buf[512];
  ...
  strcpy(buf,argc[1]);
  return(0);
}
```

- The Stack Pointer is moved to the Frame Pointer
- The old Frame Pointer is popped from the stack
- The old value was overwritten, the location of the FP is therefore meaningless
- The return address is popped from the stack into the Instruction Register (IR)

Buffer Overflow on the stack




```
int something(int,int){
  char buf[512];
  ...
  strcpy(buf,argc[1]);
  return(0);
}
```

- The new return address points to a location in the buffer
- The program executes the NOPs and the following shellcode

Exploiting a Buffer Overflow (II)

Small errors can have a profound effect

- "Of by One" errors
  - The buffer size is miscalculated by one
  - Example: `a[9]` has 10 elements
  - Result: possible manipulation of the frame pointer

Buffer Overflows on the heap are as dangerous as on the stack

- Function pointer manipulation
- Overwriting of C++ virtual pointer tables
- Exploitation of heap management functions
  - Highly dependent on the operating system
  - Sometimes in connection with further programming mistakes like for example "Double Free Errors"

Causes

Causes for Buffer Overflows (I)

- **Library functions, that don't check buffer boundaries:**
  `strcpy(), strcat(), sprintf(),…`
- **Library functions, that rely on the programmer to pass correct buffer boundaries:**

```
char buf1[4];
char buf2[8];
buf2 = "1234567";
strncpy(buf1,buf2,8);
```

- **Careless pointer arithmetic:**

```
while(<some condition>){
    *buf1++ = *buf2++
}
```

  (cause of the Blaster Worm)
- **Careless type casting of variables**

Programming Techniques

- **Don't use C, C++ or Objective C**

- Use "safer" library replacements
  - String libraries:
    - strsafe (Microsoft)
    - strlcat/strlcpy (Unix)
  - Integer:
    - SafeInt (Microsoft)
- Replace C Strings with C++ Strings
- C++: Replace static arrays with STL containers
- Avoid pointer arithmetic if possible

Thank you
for your attention

Bibliography (Buffer Overflows)

- Michael Howard: “Fix Those Buffer Overruns!”, 2002, http://msdn.microsoft.com/library/default.asp?url=/library/en-us/dncode/html/secure05202002.asp
- Michael Howard: “Reviewing Code for Integer Manipulation Vulnerabilities”, 2003, http://msdn.microsoft.com/library/default.asp?url=/library/en-us/dncode/html/secure04102003.asp
- Michael Howard: “An Overlooked Construct and an Integer Overflow Redux“, 2003, http://msdn.microsoft.com/library/default.asp?url=/library/en-us/dncode/html/secure09112003.asp
- Aleph One: "Smashing The Stack For Fun And Profit", Phrack 49, Volume 7, http://www.phrack.org/phrack/49/P49-14
- Matt Conover: "woowoo on Heap Overflows", http://www.woowoo.org/files/articles/heaptut.txt
- Rix: "Smashing C++ VPTRS", Phrack 56, Volume 11, http://www.phrack.org/phrack/56/p56-0x08
- J.C. Foster, V. Osipov, N. Bhella, N. Heinen: “Buffer Overflow Attacks”, Syngress, 2005

Bibliography (Programming)

- Martyn Lovell: „Repel Attacks on Your Code with the Visual Studio 2005 Safe C and C++ Libraries“, 2005, http://msdn.microsoft.com/msdnmag/issues/05/05/SafeCandC/default.aspx
- Michael Howard: „Strsafe.h: Safer String Handling in C“, 2002, http://msdn.microsoft.com/library/default.asp?url=/library/en-us/dnsecure/html/strsafe.asp
- David LeBlanc: „Integer Handling with the C++ SafeInt Class“, 2004, http://msdn.microsoft.com/library/default.asp?url=/library/en-us/dncode/html/secure01142004.asp
- Todd C. Miller and Theo de Raadt: „strlcpy and strlcat - consistent, safe, string copy and concatenation“, 1999 USENIX Annual Technical Conference http://www.usenix.org/events/usenix99/full_papers/millert/millert.pdf

Bibliography (static tools)

- **BOON:** D.Wagner, J. Foster, E. Brewer, and A. Aiken. A first step towards automated detection of buffer overrun vulnerabilities. In *Proceedings of NDSS 2000*, 2000.
  http://www.cs.berkeley.edu/~daw/papers/overruns-ndss00.ps
- **CQUAL:** U. Shankar, K. Talwar, J. S. Foster, and D. Wagner. Detecting format string vulnerabilities with type qualifiers. In Proceedings of the 10th USENIX Security Symposium, 2001.
  http://www.cs.berkeley.edu/~ushankar/research/percents/percents.pdf
- **Flawfinder:** David A. Wheeler. Flawnder. Web page http://www.dwheeler.com/flawfinder/, May 2001.
- **ITS4:** John Viega, J.T. Bloch, Tadayoshi Kohno, and Gary McGraw. ITS4: A static vulnerability scanner for C and C++ code. In Proceedings of the 16th Annual Computer Security Applications Conference, December 2000
  http://www.cigital.com/papers/download/its4.pdf
- **MOPS**: H. Chen and D. Wagner. MOPS: an Infrastructure for Examining Security Properties of Software. In *Proceedings of the ACM Computer and Communications Security (CCS) Conference*, pages 235–244, November 2002.
  http://www.cs.berkeley.edu/~daw/papers/mops-ccs02.ps
- **Splint:** D. Larochelle and D. Evans. Statically Detecting Likely Buffer Overflow Vulnerabilites. In *Proceedings of the 10th USENIX Security Symposium*, August 2001.
  http://lclint.cs.virginia.edu/usenix01.pdf
- **XGCC:** Seth Hallem, Benjamin Chelf, Yichen Xie, and Dawson Engler. A system and language for building system-specific static analyses. In *ACM Conference on Programming Language Design and Implementation*, pages 69--82, Berlin, Germany, June 2002
  http://www.stanford.edu/~engler/p27hallem.ps

Bibliography (dynamic tools)

- **STOBO:** E. Haugh and M. Bishop, "Testing C Programs for Buffer Overflow Vulnerabilities," *Proceedings of the 2003 Symposium on Networked and Distributed System Security* (Feb. 2003). http://nob.cs.ucdavis.edu/~bishop/papers/2003-bufov/index.html
- **CRED**: O. Ruwase and M. S. Lam. A Practical Dynamic Buffer Overflow Detector. In Proceedings of the Network and Distributed System Security (NDSS) Symposium, pages 159--169, February 2004. http://suif.stanford.edu/papers/tunji04.pdf
- **StackGuard**: C. Cowan, C. Pu, D. Maier, H. Hinton, J. Walpole, P. Bakke, S. Beattie, A. Grier, P. Wagle, and Q. Zhang. Stackguard: Automatic adaptive detection and prevention of buffer-overflow attacks. In Proceedings of the 7th USENIX Security Symposium, pages 63--78, San Antonio, TX, January 1998. http://citeseer.ist.psu.edu/cowan98stackguard.html
- **StackShield**: http://www.angelfire.com/sk/stackshield/
- **ProPolice SSP**: http://www.research.ibm.com/trl/projects/security/ssp/
- **CCured**: G. C. Necula, S. McPeak, and W. Weimer. CCured: Type-safe retrofitting of legacy code. In Twenty-Ninth ACM Symposium on Principles of Programming Languages, Portland, OR, Jan. 2002. http://manju.cs.berkeley.edu/ccured/popl02.pdf

Bibliography (Comparisons)

- J. Wilander, M. Kamkar. A Comparison of Publicly Available Tools for Dynamic Buffer Overflow Prevention. In *Proceedings of the 10th Network and Distributed System Security Symposium*, San Diego, CA, February 2003, pp. 149-162
  http://www.ida.liu.se/~johwi/research_publications/paper_ndss2003_john_wilander.pdf
- J. Wilander and M. Kamkar. A comparison of publicly available tools for static intrusion detection. In *Proceedings of the Nordic Workshop on Secure IT Systems*, pages 68--84, November 2002.
  http://www.ida.liu.se/~johwi/research_publications/paper_nordsec2002_john_wilander.pdf
- Peter Silberman, Richard Johnson (iDefense). A Comparison of Buffer Overflow Prevention Implementations and Weaknesses. *Black Hat USA 2004 Briefings & Training,* 2004
  http://www.blackhat.com/presentations/bh-usa-04/bh-us-04-silberman/bh-us-04-silberman-paper.pdf
- Yves Younan, Wouter Joosen and Frank Piessens. Code injection in C and C++: A Survey of Vulnerabilities and Countermeasures. *Technical Report CW386*, Departement Computerwetenschappen, Katholieke Universiteit Leuven, July 2004.
  http://fort-knox.org/CW386.pdf